TermXT scripting language user manual

Current version: **1.0.2**.

TermXT runs `.xt` scripts inside xTerminal. A script can use variables, conditions, loops, functions, file operations and xTerminal commands. This is the current manual. The 1.0.0 PDF is retained as an older reference.

Contents

Running and checking scripts

```
xt script.xt
xt "scripts with spaces\report.xt" -p production 8080
xt -new script.xt
xt -edit script.xt
xt -check script.xt
xt -ver
xt -h
```

`-new` writes a template to the specified file, overwriting an existing file. `-edit` opens the built-in editor; `xte script.xt` also opens it. Scripts are checked for structural errors before execution. `-check` checks without executing commands and reports failure through xTerminal's command error state. Structural checks include unmatched blocks, invalid branch ordering, missing assignments, empty variable names, duplicate functions and misplaced `break`, `continue` or `return`. The editor uses the same structural checks and also provides function-call diagnostics and keyword suggestions.

A successful check does not guarantee that expressions, commands or file paths will succeed at runtime.

Arguments and variables

Arguments after `-p` become `{1}`, `{2}`, and so on. Double quotes keep an argument containing spaces together. `""` passes an empty argument, which still counts toward `{argc}`. A quoted `-p` inside a path or argument is literal text. Spaces and tabs can separate arguments and language keywords.

```
# xt greet.xt -p "Ada Lovelace"
set name = reader
if {argc} >= 1
    set name = {1}
end
print "Hello {name}"
```

Use `{argc}` to test whether an argument was supplied. Missing variables stay literal: an undefined `{1}` produces the text `{1}`. Comparing `{1}` with itself, or with `null`, does not detect a missing argument. `null` is ordinary text.

```
set name = Ada
set message = "Hello {name}"
print "{message}"
```

Variable names and language keywords are case-insensitive. Names consist of word characters (letters, digits and underscores). Values are text; `set` creates or replaces a variable. Variables other than positional arguments are shared across function calls. Lines beginning with `#`, after optional indentation, are comments. Blank lines are ignored. Comments occupy their own lines.

Variable	Value
`{1}`, `{2}`, ...	Arguments of the current script or function call.
`{argc}`	Number of arguments in the current script or function call.

Variable	Value
{DATE}	Current date, yyyy-MM-dd.
{TIME}	Current time, HH:mm:ss.
{USER}	Current account name.
{PC}	Computer name.
{CWD}	Current xTerminal working directory.
{i}	Current loop iteration, starting at 1. Nested loops restore the outer counter.
{result}	Most recent return value; cleared when entering a function call.
{error}	true following a failed operation; false after a successful command or file operation, or a handled try/catch.
{error_message}	Most recent error message; available inside catch.

DATE, TIME and CWD are evaluated when used. Interpolation is a single pass: placeholder-shaped text inside a substituted value is not expanded again.

Text and arithmetic

```
set total = eval (10 + 2) * 3
set remainder = eval 10 % 3
set loud = upper hello world
set quiet = lower HELLO WORLD
set size = len "hello world"
set clean = trim "  hello  "
set part = substr "hello world" 6 5
set changed = replace "hello world" "world" "TermXT"
print "{total}: {changed}"
```

eval supports numbers, decimal points, +, -, *, /, % and parentheses. Results use a decimal point regardless of the Windows number format. Invalid expressions and non-finite results, including division by zero, report an error and enter an enclosing catch. Outside a try, failed arithmetic reports the error and supplies 0; inspect {error} before using that value.

substr uses a zero-based start and a nonnegative length. Negative starts are clamped to zero, lengths are clamped to the remaining text, and a start beyond the text returns an empty string. Negative lengths report an error. replace matches text without regard to case. Quote text arguments that contain spaces; use "" for an empty replacement.

print adds a newline. It recognizes \n, \t and \\ escape sequences, including those stored in variables. Absolute Windows drive paths and UNC paths are preserved by its path handling.

Conditions

```
if {count}>=2&&{count}<5
    print "Between two and four"
elif {count} == 5
    print "Exactly five"
else
    print "Outside the range"
end

if not {error} && ({mode} == quick || {mode} == full)
    print "Ready"
end

if "ready||waiting" contains "||"
    print "The operator is part of the text"
end
```

Operator	Meaning
`==`, `!=`	Case-insensitive text equality or inequality.
`>`, `<`, `>=`, `<=`	Numeric comparison when both values parse as numbers; otherwise case-insensitive text ordering.
`contains`	Left text includes right text.
`startswith`, `endswith`	Left text has the specified prefix or suffix.
`not`	Negates the following comparison or grouped condition.
`&&`, `||`	Logical AND and OR, with short-circuit evaluation.

Precedence is parentheses, comparisons/`not`, `&&`, then `||`. For example, `not false && false` is false; use `not (false && false)` to negate the whole expression. Symbolic operators can touch their operands; word operators such as `contains` require whitespace around them. Conditions can be nested up to the interpreter's evaluation depth limit of 128.

Operators and parentheses inside double-quoted text are literal. Substituted variable values are treated as data, even if they contain operators, quotes or newlines. Undefined placeholders retain their literal value. For a truthy test, empty text, `0` and `false` are false; other text is true. Equality is textual, so `1` and `1.0` are different for `==`. Write an empty comparison operand as `""`.

Each `if` allows multiple `elif` branches, followed by at most one `else`. Neither `elif` nor another `else` can follow `else`.

Loops

```
loop 3
    print "Iteration {i}"
end

set remaining = 3
while {remaining} > 0
    print "{remaining}"
    set remaining = eval {remaining} - 1
end

each color in red,green,blue
    print "{i}: {color}"
end

each n in 3..1
    print "{n}"
end

read report = "report.txt"
each line in lines:report
    print "{line}"
end
```

`loop` requires a positive integer. Numeric `each` ranges are inclusive, support ascending and descending order, and accept signed 32-bit endpoints. A range at either integer limit terminates without wrapping. Comma-separated iteration skips empty items and trims surrounding whitespace; it is not a CSV parser for quoted commas. `lines:report` and `lines:{report}` iterate nonempty lines of a variable, trimming each line.

`break` exits the innermost loop; `continue` skips to its next iteration. `return` inside a loop exits its function. A function cannot use `break` or `continue` to control a caller's loop. An inner loop restores the outer `{i}`, including when an error leaves the inner loop. `wait 1000` pauses for one second. `exit` stops the entire script, including when used in a function.

Functions

```
func greet
    return Hello {1}
end

set person = "Ada Lovelace"
call greet {person}
print "{result}"
```

Functions may be called before their definition. Function names are case-insensitive and duplicate definitions are rejected. `call` parses arguments before substituting variables, so `{person}` remains one argument when its value contains whitespace or quotes. Literal multiword arguments still need quotes.

Each call has its own positional arguments and `{argc}`. Caller arguments are restored on return and on failure, without a 20-argument limit. New arguments do not leak into the caller. Other variables remain shared. `return value` stores text in `{result}` and ends the function; bare `return` stores empty text. Call nesting is limited to 128 to prevent runaway recursion from crashing xTerminal.

Commands, pipes and errors

```
run time
capture files = ls
capture selected = ls | cat -s exe
print "{selected}"

try
    read settings = "settings.txt"
    print "Loaded settings"
catch
    print "Could not load settings: {error_message}"
end
print "Continuing"
```

Ordinary xTerminal commands can also be written without `run`. `capture` stores standard output, trimming its surrounding whitespace. A single | outside double quotes separates pipeline stages. Quoted pipes remain part of an argument; || is not split into pipeline stages.

Inside `try`, the first failed script operation, command failure flag or thrown exception stops that block and enters `catch`. Later pipeline stages also stop after a failure. `{error}` is `true` on entry to the handler and `{error_message}` describes the failure. If the handler succeeds, execution continues after `end` with `{error}` set to `false`. A handler failure propagates to an enclosing handler. A `try` without a `catch` propagates a failure to an enclosing handler, or leaves the error state set when no handler exists.

Starting a `try` clears the previous error state. Successful commands and file operations clear it as well. Simple statements such as `print` do not clear a previous failure. Outside `try`, reported script errors set `{error}` and the xTerminal command error flag; unhandled exceptions stop execution.

Input and files

```
input name = "Your name"
write "output report.txt" "Hello {name}"
append "output report.txt" "Generated on {DATE} at {TIME}"
read report = "output report.txt"
print "{report}"
```

Paths are relative to xTerminal's current working directory, unless absolute. Use double quotes for paths with spaces. `read` loads the whole file. `write` overwrites or creates a file; `append` adds to it or creates it. Both add a newline after the supplied text. `write "empty line.txt" ""` writes one empty line. Parent directories must already exist. File-write text supports interpolation and escape sequences in the script literal; substituted file contents are preserved.

Examples and changes in 1.0.2

The example directory contains `greet.xt`, `countdown.xt`, `portcheck.xt`, `sysreport.xt`, `netaudit.xt`, and the new `language_features.xt`.

```
xt Documents\TermXT_Examples\language_features.xt -p "Ada Lovelace"
```

This local demo creates `TermXT demo output.txt` in the current directory and demonstrates optional arguments, functions, compact/grouped conditions, quoted paths and recovery from a deliberate arithmetic error.

Version 1.0.2 adds `{argc}`, grouped conditions and compact symbolic condition operators. It fixes quoted logical text crashes, `not` precedence, repeated condition interpolation, tab-separated keywords, quoted `-p` parsing, function argument leakage, exception cleanup, stale return values at call entry, missed `try/catch` failures, quoted file paths, nested iteration counters and integer range overflow. Structural validation is shared with the editor. The countdown, port checker and network audit examples now use `{argc}` for their defaults.

When updating older scripts, use `{argc}` for optional arguments, put empty comparison operands in quotes, and use parentheses if `not` should negate a compound condition. Function-call argument substitution now preserves each variable value as one argument.